

Build: Survival Engine

Objective

Create a deterministic **Survival Engine** that answers:

“Can this business continue operating without running out of cash?”

It reads:

- cash balance
- revenue
- expenses
- fixed costs
- variable costs
- net cashflow
- historical cashflow snapshots

It writes structured output for:

- Master Aggregator
- Fragility Index
- Scenario Simulation
- Adaptive Scenario Intelligence
- AI Narratives
- PDF Reports

Recommended Folder

```
/src/lib/engines/survival
```

Suggested structure:

```
/src/lib/engines/survival  
├─ types.ts
```

- |— rules.ts
- |— survival-engine.ts
- |— survival-engine.test.ts
- |— index.ts

1. Input Contract

```
export interface SurvivalEngineInput {
  businessId: string;

  financials: {
    cashBalance: number;
    monthlyRevenue: number;
    monthlyExpenses: number;
    variableCostsMonthly: number;
    fixedCostsMonthly: number;
    netCashflowMonthly?: number;
  };

  history?: {
    revenueLast6Months?: number[];
    expensesLast6Months?: number[];
    netCashflowLast6Months?: number[];
  };

  liquidity?: {
    creditAvailable?: number;
    overdraftLimit?: number;
    committedFutureRevenue?: number;
  };

  metadata?: {
    source?: "manual" | "imported" | "integration";
    confidenceLevel?: "LOW" | "MEDIUM" | "HIGH";
    rulesetVersion?: string;
  };
}
```

2. Output Contract

```
export interface SurvivalEngineOutput {  
  engine: "survival";  
  version: string;  
  businessId: string;  
  
  metrics: {  
    cashBalance: number;  
    monthlyRevenue: number;  
    monthlyExpenses: number;  
    netCashflowMonthly: number;  
    burnRate: number;  
    runwayMonths: number | null;  
    volatilityRatio: number | null;  
    liquidityBuffer: number;  
  };  
  
  scores: {  
    runwayScore: number;  
    burnScore: number;  
    volatilityScore: number;  
    survivalScore: number;  
  };  
  
  classification: {  
    scenario:  
      | "STRONG_SURVIVAL"  
      | "STABLE_BUT_WATCH"  
      | "CASH_PRESSURED"  
      | "HIGH_BURN_RISK"  
      | "CRITICAL_SURVIVAL_RISK";  
    label: string;  
  };  
  
  riskFlags: Array<{
```

```
code: string;
severity: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL" | "VERY_HIGH";
explanation: string;
recommendedAction: string;
priorityScore: number;
}>;

recommendations: Array<{
  type: string;
  priority: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
  text: string;
  expectedImpact?: string;
}>;

interpretation: {
  headline: string;
  driverSummary: string;
  businessInterpretation: string;
  riskOutlook: string;
  recommendedActions: string[];
};

confidence: {
  level: "LOW" | "MEDIUM" | "HIGH";
  note: string;
  penaltyApplied: boolean;
};

trace: Record<string, unknown>;
}
```

3. Core Calculations

Use the shared Computation Library.

Net Cashflow

```
net_cashflow_monthly =  
monthly_revenue - monthly_expenses
```

If netCashflowMonthly is provided, use it.
If not provided, derive it.

Burn Rate

```
burn_rate =  
if net_cashflow_monthly < 0  
then abs(net_cashflow_monthly)  
else 0
```

Cash Runway

```
runway_months =  
if burn_rate == 0  
then null  
else cash_balance / burn_rate
```

Use null instead of infinity for API safety.

Cashflow Volatility

```
volatility_ratio =  
standard_deviation(net_cashflow_last_6_months) /  
abs(mean(net_cashflow_last_6_months))
```

Rules:

- If fewer than 2 months of history, return null
- If mean is 0, return null

- If no history, apply confidence penalty

Liquidity Buffer

```
liquidity_buffer =  
cash_balance + credit_available + overdraft_limit +  
committed_future_revenue
```

If optional fields are missing, treat as 0.

4. Scoring Rules

Runway Score

```
if runwayMonths === null && burnRate === 0 => 100  
else if runwayMonths > 12 => 100  
else if runwayMonths >= 6 => 70  
else if runwayMonths >= 3 => 40  
else => 10
```

Burn Score

```
if burnRate === 0 => 100  
else if burnRate < 0.2 * monthlyRevenue => 70  
else if burnRate <= 0.5 * monthlyRevenue => 40  
else => 10
```

If monthlyRevenue <= 0 and burnRate > 0:

```
burnScore = 10
```

Volatility Score

```
if volatilityRatio === null => 50
else if volatilityRatio < 0.5 => 100
else if volatilityRatio <= 1.0 => 70
else if volatilityRatio <= 2.0 => 40
else => 10
```

Fallback:

```
volatilityScore = 50
confidencePenaltyApplied = true
```

5. Final Survival Score

```
survival_score =
0.40 × runway_score +
0.30 × burn_score +
0.30 × volatility_score
```

6. Classification Logic

Priority order matters.

```
if runwayScore <= 10 || survivalScore < 40:
    CRITICAL_SURVIVAL_RISK

else if burnScore <= 40 && runwayScore <= 70:
    HIGH_BURN_RISK

else if runwayScore <= 50 && burnScore <= 50:
    CASH_PRESSURED

else if runwayScore >= 85 && burnScore >= 70 && survivalScore >= 80:
    STRONG_SURVIVAL
```

else:

 STABLE_BUT_WATCH

Labels:

STRONG_SURVIVAL = "Strong Survival"

STABLE_BUT_WATCH = "Stable but Watch"

CASH_PRESSED = "Cash Pressured"

HIGH_BURN_RISK = "High Burn Risk"

CRITICAL_SURVIVAL_RISK = "Critical Survival Risk"

7. Risk Flags

CRITICAL_RUNWAY_RISK

Trigger:

runwayMonths !== null && runwayMonths < 3

Severity:

VERY_HIGH

Action:

Reduce costs immediately, increase short-term cash inflows, or secure liquidity.

LOW_RUNWAY_RISK

Trigger:

runwayMonths !== null && runwayMonths >= 3 && runwayMonths < 6

Severity:

HIGH

Action:

Improve cash reserves, reduce burn, or accelerate revenue collection.

HIGH_BURN_RATE

Trigger:

`monthlyRevenue > 0 && burnRate > 0.5 * monthlyRevenue`

Severity:

HIGH

Action:

Reduce recurring expenses or improve monthly revenue before adding fixed commitments.

NEGATIVE_CASHFLOW

Trigger:

`netCashflowMonthly < 0`

Severity:

MEDIUM

Action:

Review spending, improve collections, and increase near-term inflows.

VOLATILE_CASHFLOW

Trigger:

`volatilityScore <= 40`

Severity:

HIGH

Action:

Stabilise revenue, reduce unpredictable expenses, and improve recurring income.

NO_CASH_VISIBILITY

Trigger:

netCashflowLast6Months missing or length < 2

Severity:

MEDIUM

Action:

Add at least 3–6 months of revenue, expense, and cashflow history to improve accuracy.

8. Recommendations Logic

Generate top 3 recommendations based on risk priority.

Priority order:

1. CRITICAL_RUNWAY_RISK
2. LOW_RUNWAY_RISK
3. HIGH_BURN_RATE
4. NEGATIVE_CASHFLOW
5. VOLATILE_CASHFLOW
6. NO_CASH_VISIBILITY

Example recommendations:

```
[
  {
    type: "cash_preservation",
    priority: "CRITICAL",
    text: "Reduce non-essential expenses immediately to extend
runway."
  },
  {
    type: "revenue_collection",
    priority: "HIGH",
    text: "Accelerate customer payments and improve short-term cash
inflows."
  },
  {
    type: "cash_visibility",
    priority: "MEDIUM",
    text: "Add 6 months of cashflow history to improve survival
confidence."
  }
]
```

9. Interpretation Rules

STRONG_SURVIVAL

The business shows strong short-term financial resilience.

STABLE_BUT_WATCH

The business is currently stable, but cash discipline should be maintained.

CASH_PRESSED

The business is under cash pressure and needs corrective action.

HIGH_BURN_RISK

The business is burning cash at a rate that may become dangerous if not corrected.

CRITICAL_SURVIVAL_RISK

The business is at immediate risk of running out of cash.

10. Confidence Logic

Use this order:

```
if metadata.confidenceLevel exists:
    use it
else if metadata.source === "integration":
    HIGH
else if history.netCashflowLast6Months length >= 3:
    MEDIUM
else:
    LOW
```

Apply penalty if:

```
history.netCashflowLast6Months missing or length < 2
```

Confidence notes:

HIGH: Based on verified or integrated financial data.

MEDIUM: Based on partial history and structured assumptions.

LOW: Based mainly on estimates and should be validated with real cashflow data.

11. Trace Object

Include:

```
trace: {
  formulas: {
    netCashflowMonthly: "monthlyRevenue - monthlyExpenses",
    burnRate: "netCashflow < 0 ? abs(netCashflow) : 0",
    runwayMonths: "cashBalance / burnRate",
    volatilityRatio: "std_dev(netCashflowLast6Months) / abs(mean)"
  },
  inputsUsed: input,
  scoringRules: {
    runwayWeight: 0.4,
    burnWeight: 0.3,
    volatilityWeight: 0.3
  }
}
```

12. Unit Tests

Create tests for:

Strong Survival Case

Input:

- cash balance 50,000
- monthly revenue 20,000
- monthly expenses 15,000
- positive cashflow
- stable 6-month cashflow

Expected:

- burnRate = 0

- runwayMonths = null
- runwayScore = 100
- survivalScore high
- classification STRONG_SURVIVAL

Critical Runway Case

Input:

- cash balance 3,000
- revenue 10,000
- expenses 13,000

Expected:

- burnRate = 3,000
- runwayMonths = 1
- CRITICAL_RUNWAY_RISK
- CRITICAL_SURVIVAL_RISK

High Burn Case

Input:

- revenue 10,000
- expenses 17,000

Expected:

- burnRate = 7,000
- HIGH_BURN_RATE

No History Case

Input:

- no netCashflowLast6Months

Expected:

- volatilityScore = 50
- NO_CASH_VISIBILITY
- confidence penalty applied

Volatile Cashflow Case

Input:

- net cashflow values with high variation

Expected:

- VOLATILE_CASHFLOW

13. Acceptance Criteria

Codex should complete this when:

- Survival Engine accepts the defined input contract
- It uses computation library functions
- It returns the defined output contract
- All risk flags are generated correctly
- Recommendations are prioritised
- Interpretation is deterministic
- Confidence is included
- Trace is included
- Unit tests pass
- No database calls inside engine
- No LLM calls inside engine

14. Codex Prompt

Build the EnterprateAI Survival Engine in `/src/lib/engines/survival`.

The engine must be a pure deterministic TypeScript module. It must not call the database, API, or LLM.

It should accept `SurvivalEngineInput` and return `SurvivalEngineOutput`.

Use the shared computation library from `/src/lib/computation` for:

- burn rate
- runway months
- volatility ratio
- weighted score

Implement:

- net cashflow derivation
- liquidity buffer calculation
- runway score
- burn score
- volatility score
- final survival score
- scenario classification using priority rules
- risk flags
- top 3 recommendations
- interpretation
- confidence layer
- trace object

Create:

- `types.ts`
- `rules.ts`
- `survival-engine.ts`
- `survival-engine.test.ts`
- `index.ts`

Add unit tests for:

- strong survival case

- critical runway case
- high burn case
- no history case
- volatile cashflow case

Export the engine from index.ts.